Voice Orchestrator — Mobile Developer Handoff

Karthi Jeyabalan

2026-04-28

Voice Orchestrator — Mobile Developer Handoff

Everything you need to integrate the Voice Orchestrator API into the SmartHome mobile app. All examples are curl — port them to your platform's HTTP client of choice.

---

1. Credentials

Item	Value
API base URL	https://voiceorchestrator.homeadapt.us/api/v1/voice-auth
VAPI assistant id	1a2904b1-61cf-49da-a804-199d8d39fb9f
VAPI public key (browser/SDK only)	0145170b-662c-4db1-983d-b3bf8aeee1b4
Mobile API key — iOS	sk_ios_48b546d143dae04ec9d2c4396ae9155648e80a5ffbaf3377
Mobile API key — Android	sk_and_a4680b4e40ffe9b5133e8118b7d41cf753fa062688bc05a0
Mobile API key — Web	sk_web_37ceccf66f52a679b34ce1ba53eca44d7f67f2c281d15bc1

> Keys are static for v1. Compile via your CI secret store; never commit to git. Tomorrow they become short-lived JWTs — header shape stays `Authorization: Bearer <token>`, only the token source moves.

---

2. Authentication

Every REST request needs:

```
Authorization: Bearer <your-platform-key>
```

Missing or wrong key → `401 { "error": "...", "code": "UNAUTHORIZED" }`.

Examples below use a shell variable:

```
KEY=sk_ios_48b546d143dae04ec9d2c4396ae9155648e80a5ffbaf3377
BASE=https://voiceorchestrator.homeadapt.us/api/v1/voice-auth
```

---

3. Endpoint Reference

The API has five feature groups:

1. **Voice-auth enrollments** — gate an automation behind a voice phrase challenge
2. **Favorites** — per-user pinned HA entities for the dashboard
3. **Scene mappings** — register HA webhook scenes
4. **Automations trigger** — fire a scene/script directly (with voice-gate guard)
5. **Voice enable** — provision a VAPI phone number for a user

Plus the **VAPI SDK** for the spoken challenge UX.

3.1 Voice-Auth Enrollments

User toggles “Voice Protect” on a scene → enrollment row created. After that, direct triggers for that scene are blocked until the user passes a voice challenge.

Create an enrollment

```
curl -s -X POST "$BASE/enrollments" \
  -H "Authorization: Bearer $KEY" \
  -H "Content-Type: application/json" \
  -d '{
    "user_ref":        "scott_mobile",
    "home_id":         "scott_home",
    "automation_name": "Main Lights On",
    "ha_service":      "script",
    "ha_entity":       "main_lights_on"
  }'
```

ha_service ∈ {scene, script, switch, light, lock, cover, media_player, climate, input_boolean, fan}. ha_entity is the **suffix only** — main_lights_on, **not** script.main_lights_on.

Response 201:

```
{
  "id":              "a8100373-7f6c-4e27-9ecf-71d19c1ed4cd",
  "user_ref":        "scott_mobile",
  "home_id":         "scott_home",
  "automation_id":   "main_lights_on",
  "automation_name": "Main Lights On",
  "ha_service":      "script",
  "ha_entity":       "main_lights_on",
  "status":          "ACTIVE",
  "max_attempts":    3,
  "cooldown_seconds": 30
}
```

Idempotent on (user_ref, automation_id).

List enrollments

```
curl -s -H "Authorization: Bearer $KEY" \
  "$BASE/enrollments?user_ref=scott_mobile"
```

Response 200:

```
{
  "count": 1,
  "items": [ { "id": "a8100373-…", "automation_name": "Main Lights On", "status": "ACTIVE" } ]
}
```

Pre-flight check before launching VAPI session

```
curl -s -H "Authorization: Bearer $KEY" \
  "$BASE/check?user_ref=scott_mobile&automation_id=main_lights_on"
```

Response 200:

```
{
  "exists":              true,
  "enrollment_id":       "a8100373-…",
  "status":              "ACTIVE",
  "cooldown_remaining_seconds": 0,
  "attempts_remaining":  3,
  "enrollment_required": false
}
```

If exists: false → user hasn’t enrolled this automation; show enrollment UI. If cooldown_remaining_seconds > 0 → display countdown, block voice session.

Pause / resume / revoke

```
# Pause
curl -s -X PATCH "$BASE/enrollments/<id>/status" \
  -H "Authorization: Bearer $KEY" -H "Content-Type: application/json" \
  -d '{"status":"PAUSED"}'

# Resume
curl -s -X PATCH "$BASE/enrollments/<id>/status" \
  -H "Authorization: Bearer $KEY" -H "Content-Type: application/json" \
  -d '{"status":"ACTIVE"}'

# Permanently revoke (returns 204)
curl -s -X DELETE "$BASE/enrollments/<id>" -H "Authorization: Bearer $KEY"
```

Recent challenge attempts (audit log)

```
curl -s -H "Authorization: Bearer $KEY" \
  "$BASE/challenges?user_ref=scott_mobile&limit=10"
```

Each entry records `result` (PASS / FAIL / TIMEOUT) and `vapi_call_id` for correlating with VAPI dashboards.

3.2 Favorites

Per-user pinned HA entities. Drives the home-screen tile grid.

Discover candidates from Home Assistant

```
curl -s -H "Authorization: Bearer $KEY" \
  "$BASE/automations/discover?home_id=scott_home"
```

Returns every voice-eligible entity (lights, scenes, scripts, switches, locks, covers, media players, climate, fans) with friendly_name + state. Use this to populate the “Add favorite” picker.

Add a favorite

```
curl -s -X POST "$BASE/favorites" \
  -H "Authorization: Bearer $KEY" -H "Content-Type: application/json" \
  -d '{
    "user_ref":      "scott_mobile",
    "home_id":       "scott_home",
    "entity_id":     "light.kitchen",
    "friendly_name": "Kitchen Lights"
  }'
```

`entity_id` must be of the form `domain.suffix`. `friendly_name` is optional; if omitted, the suffix is used. `position` is optional; defaults to next-in-list.

Response `201`:

```
{
  "id":           "70640f1e-9702-4bd7-8e02-05fd2dffb614",
  "user_ref":     "scott_mobile",
  "home_id":      "scott_home",
  "entity_id":    "light.kitchen",
  "friendly_name": "Kitchen Lights",
  "domain":       "light",
  "position":     0,
  "created_at":   "2026-04-28T15:30:09.818419"
}
```

Duplicate (user_ref, home_id, entity_id) → `400`.

List favorites

```
curl -s -H "Authorization: Bearer $KEY" \
  "$BASE/favorites?user_ref=scott_mobile&home_id=scott_home"
```

Response 200:

```
{
  "count": 2,
  "items": [
    { "id": "70640f1e-…", "entity_id": "light.kitchen",     "position": 0 },
    { "id": "eee717e4-…", "entity_id": "scene.movie_night", "position": 1 }
  ]
}
```

Items are returned ordered by position.

Remove a favorite

```
curl -s -X DELETE "$BASE/favorites/<id>" -H "Authorization: Bearer $KEY"
```

Returns 204 on success, 404 if id is unknown.

Reorder (drag-and-drop UI)

```
curl -s -X PATCH "$BASE/favorites/reorder" \
  -H "Authorization: Bearer $KEY" -H "Content-Type: application/json" \
  -d '{
    "items": [
      {"id": "eee717e4-…", "position": 0},
      {"id": "70640f1e-…", "position": 1}
    ]
  }'
```

Send the full new order. Response is the updated list.

3.3 Scene Mappings

Map a friendly scene name (e.g. "decorations on") to an HA webhook id. This is what lets Alexa or VAPI translate spoken names into webhook calls.

Create a scene mapping

```
curl -s -X POST "$BASE/scene-mappings" \
  -H "Authorization: Bearer $KEY" -H "Content-Type: application/json" \
  -d '{
    "home_id":    "scott_home",
    "scene_name": "Movie Night",
    "webhook_id": "movie_night_1751404299018"
  }'
```

Scene names are normalized to lowercase server-side.

Response 201:

```
{
  "id":         "3e849ff7-be41-4bd2-aae4-eb139f3a0c9b",
  "home_id":    "scott_home",
  "scene_name": "movie night",
  "webhook_id": "movie_night_1751404299018",
  "is_active":  true,
  "created_at": "2026-04-28T15:30:09.818419"
}
```

List scenes for a home

```
curl -s -H "Authorization: Bearer $KEY" \
  "$BASE/scene-mappings?home_id=scott_home"
```

Update a scene (rename, swap webhook, or deactivate)

```
curl -s -X PATCH "$BASE/scene-mappings/<id>" \
  -H "Authorization: Bearer $KEY" -H "Content-Type: application/json" \
  -d '{"webhook_id": "new_webhook_id_here"}'
```

Body fields are all optional: `scene_name`, `webhook_id`, `is_active`.

Delete

```
curl -s -X DELETE "$BASE/scene-mappings/<id>" -H "Authorization: Bearer $KEY"
```

3.4 Automation Trigger

Direct fire-and-forget activation. Used when the user taps a favorite tile or a "scenes" button. **Refuses with 409 if a voice-auth enrollment gates the automation** — the app must then route through the VAPI session instead.

Fire an automation

```
curl -s -X POST "$BASE/automations/trigger" \
  -H "Authorization: Bearer $KEY" -H "Content-Type: application/json" \
  -d '{
    "home_id":      "scott_home",
    "ha_service":   "scene",
    "ha_entity":    "movie_night",
    "user_ref":     "scott_mobile",
    "automation_id": "movie_night"
  }'
```

`user_ref` and `automation_id` are **required** for the voice-gate check — omit them and the gate is bypassed (only do that for clearly non-protected items).

Response `200`:

```
{ "success": true, "message": "OK", "status_code": 200, "latency_ms": 142 }
```

Response `409` (voice-auth enrollment exists for this automation):

```
{
  "error":       "this automation requires voice authentication",
  "code":        "ENROLLMENT_REQUIRED",
  "enrollment_id": "a8100373-…"
}
```

When you see `409 ENROLLMENT_REQUIRED`, start the VAPI SDK with the appropriate `variableValues` (see §3.6) instead of retrying this endpoint.

Response `502` if Home Assistant is unreachable; `400` for validation errors.

3.5 Voice Enable (VAPI provisioning)

Provision a dedicated phone number for the user via VAPI. After this, the user can place outbound calls TO that number to authenticate by voice.

Enable voice for a user

```
curl -s -X POST "$BASE/voice-enable" \
  -H "Authorization: Bearer $KEY" -H "Content-Type: application/json" \
  -d '{
    "user_ref":  "scott_mobile",
    "home_id":   "scott_home",
    "area_code": "415"
  }'
```

area_code is optional; if omitted, VAPI picks one.

Response 201:

```
{
  "id":                 "432da968-9f19-4fba-b92c-36bee112c7f1",
  "user_ref":             "scott_mobile",
  "home_id":              "scott_home",
  "phone_e164":             "+14151702028",
  "vapi_phone_number_id":  "vpn_abc123…",
  "is_active":            true
}
```

Idempotent — calling again for the same (user_ref, home_id) returns the existing mapping without a second VAPI purchase. **Billable on the VAPI side** when first called.

Status check

```
curl -s -H "Authorization: Bearer $KEY" \
  "$BASE/voice-enable?user_ref=scott_mobile"
```

Response 200:

```
{
  "enabled":    true,
  "is_dry_run":  false,
  "mapping": { "phone_e164": "+14151702028", "vapi_phone_number_id": "vpn_abc123…" }
}
```

is_dry_run: true means the server hasn't been configured with a live VAPI key — useful for staging environments. Production should always show false.

Disable (release the number)

```
curl -s -X DELETE "$BASE/voice-enable?user_ref=scott_mobile" \
  -H "Authorization: Bearer $KEY"
```

Returns 204 on success, 404 if no active mapping.

3.6 VAPI SDK — Voice Challenge

When the user taps a voice-protected automation (or after a 409 ENROLLMENT_REQUIRED from /automations/trigger), launch the VAPI SDK with the public key and these variableValues:

```
assistantId: "1a2904b1-61cf-49da-a804-199d8d39fb9f"

assistantOverrides.variableValues:
  home_id:         "scott_home"
  user_ref:        "scott_mobile"
  automation_id:   "main_lights_on"
  automation_name: "Main Lights On"
  initiated_by:    "MOBILE_IOS" | "MOBILE_ANDROID"
```

The assistant says *"Confirm Main Lights On? Say the security phrase."* — the user repeats — Home Assistant fires. Subscribe to VAPI's call-end event to know when it's done; then refresh enrollment state with GET /check.

VAPI SDK references: - iOS: https://github.com/VapiAI/ios - Android: https://github.com/VapiAI/android - Web: https://github.com/VapiAI/web

The SDK uses the **VAPI public key**, not the mobile API key.

---

4. Error Reference

HTTP	Code	Meaning	App action
200 / 201 / 204	—	OK	proceed
400	VALIDATION	Bad request body	fix input, show inline error
401	UNAUTHORIZED	Missing / wrong API key	auth-config bug — do not retry blindly
404	—	Not found	varies (prompt to create, etc.)
409	ENROLLMENT_REQUIRED	Automation is voice-gated	launch VAPI session instead
409	CONFLICT	State conflict (e.g. un-revoke)	surface message
502	VAPI_ERROR	VAPI provisioning failed	retry once with backoff
502	—	Home Assistant unreachable	retry once with backoff
503	NOT_CONFIGURED	Server feature not wired up	report to backend team

Error envelope is always:

```
{ "error": "<human message>", "code": "<OPTIONAL_CODE>" }
```

---

5. Acceptance Checklist

Run through these to verify the integration end-to-end:

- [ ] GET /enrollments?user_ref=demo_user with no Authorization → 401
- [ ] Same call with valid bearer → 200 { count: 0, items: [] }
- [ ] POST /enrollments → 201 with automation_id echoed back
- [ ] GET /check?user_ref=…&automation_id=… → exists: true, attempts_remaining: 3
- [ ] GET /automations/discover?home_id=… → list of HA candidates
- [ ] POST /favorites → 201 with auto-assigned position
- [ ] GET /favorites?user_ref=…&home_id=… → ordered list
- [ ] POST /automations/trigger for an **un-enrolled** automation → 200 success: true
- [ ] POST /automations/trigger for an **enrolled** automation → 409 ENROLLMENT_REQUIRED
- [ ] Launch VAPI SDK after a 409 → assistant prompts for security phrase → HA fires
- [ ] POST /voice-enable → 201 with phone_e164; second call same payload → idempotent same id
- [ ] DELETE /voice-enable?user_ref=… → 204; status flips to enabled: false

---

6. Contact

When something misbehaves, send the backend team: - The user_ref you used - A vapi_call_id from GET /challenges?user_ref=<you> if it was a voice flow - The HTTP request + response (curl -i output is ideal)

All tool-call payloads are server-logged with redacted bodies — we can trace any specific call by id.